

Classificação — do problema ao modelo confiável

Classificação é usada quando queremos prever uma categoria: fraude ou não fraude, churn ou não churn, aprovado ou reprovado, ou ainda várias categorias ao mesmo tempo.

Neste treinamento vamos estudar não apenas algoritmos, mas principalmente **como saber se um modelo realmente pode ser utilizado**: distribuição das classes, métricas, validação, threshold, classes minoritárias, balanceamento, otimização, importância de features e interpretabilidade.

Também veremos classificação **multirrótulo**, em que uma mesma observação pode receber várias classes simultaneamente.

PROBLEMAS

Binário + Multirrótulo

Uma classe positiva ou várias labels simultâneas.

FOCO

Avaliar corretamente

Não basta treinar. Precisamos saber se o modelo é confiável.

PONTO CRÍTICO

Classes minoritárias

Accuracy alta pode esconder um modelo praticamente inútil.

SEÇÃO 01

O que é um problema de classificação?

? Pergunta

Qual é a diferença entre prever um número e prever uma categoria?

Classificação binária

DUAS POSSIBILIDADES

Exemplos: fraude/não fraude, churn/não churn, compra/não compra.

Multiclasse

UMA CLASSE ENTRE VÁRIAS

Exemplo: classificar uma imagem como gato, cachorro, cavalo ou pássaro.

Multirrótulo

VÁRIAS CLASSES AO MESMO TEMPO

Um comentário pode falar simultaneamente de atendimento, preço e produto.

Ideia principal

Antes de escolher um algoritmo precisamos definir corretamente a estrutura da variável alvo.

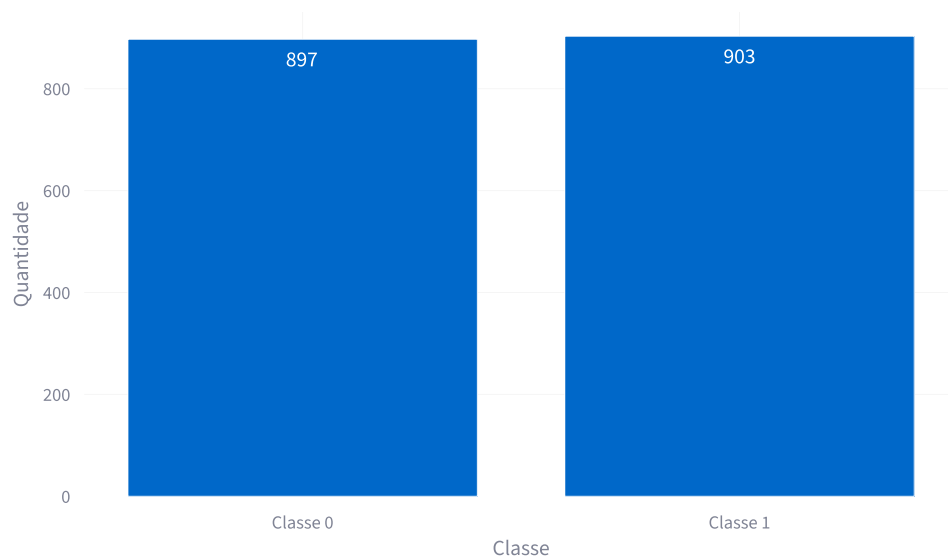
SEÇÃO 02

Classes balanceadas e desbalanceadas

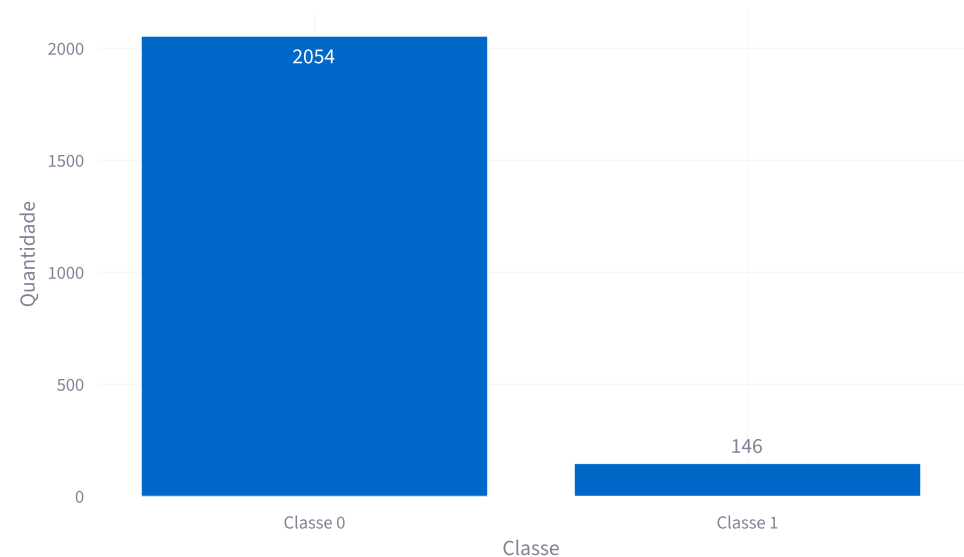
Pergunta

Se 95% dos clientes não têm churn, um modelo com 95% de accuracy é necessariamente bom?

Base balanceada



Base desbalanceada



Na base desbalanceada, prever sempre a classe majoritária já produziria 93.4% de accuracy sem aprender a classe rara.

Antes de qualquer modelo, verifique sempre:

- frequência de cada classe;
- baseline ingênuo;
- qual erro é mais caro;
- qual classe realmente precisa ser encontrada.

SEÇÃO 03

Split correto e Data Leakage

? Pergunta

Um modelo pode parecer excelente simplesmente porque recebeu informação que não deveria ter?

Treino → o modelo aprende
Validação → escolhemos parâmetros e threshold
Teste → avaliação final em dados realmente não vistos

Data Leakage

INFORMAÇÃO PROIBIDA

Acontece quando o modelo recebe direta ou indiretamente informação que só existiria depois da previsão.

Pipeline

TRANSFORMAÇÕES SEM VAZAR INFORMAÇÃO

Scaling, imputação e seleção de features devem ser aprendidos apenas com o treino de cada etapa.

💡 Ideia principal

Performance excelente com leakage é uma avaliação inválida.

SEÇÃO 04

Principais algoritmos de classificação

Modelo	Quando costuma ser útil	Ponto de atenção
Logistic Regression	Baseline forte, rápido e interpretável.	Fronteira essencialmente linear.
KNN	Quando vizinhos próximos tendem a compartilhar a classe.	Sensível à escala e ao volume.
Decision Tree	Regras simples e interpretáveis.	Pode overfitar facilmente.
Random Forest	Dados tabulares e relações não lineares.	Menos interpretável que uma árvore.
Gradient Boosting	Relações complexas em dados tabulares.	Exige mais tuning.
XGBoost / LightGBM / CatBoost	Alta performance em dados tabulares.	Dependências externas e tuning.
SVM	Fronteiras complexas em bases médias.	Pode ser pesado em bases grandes.
Naive Bayes	Texto e problemas probabilísticos simples.	Hipóteses fortes sobre os dados.

SEÇÃO 05

Comparando modelos em uma base binária

Modelo	Accuracy	Balanced Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.865	0.865	0.851	0.886	0.868	0.938	0.925
KNN	0.939	0.939	0.931	0.948	0.940	0.978	0.974
Decision Tree	0.876	0.876	0.864	0.893	0.878	0.898	0.854
Random Forest	0.937	0.937	0.928	0.948	0.938	0.975	0.975
Gradient Boosting	0.931	0.931	0.927	0.937	0.932	0.972	0.969

 Ideia principal

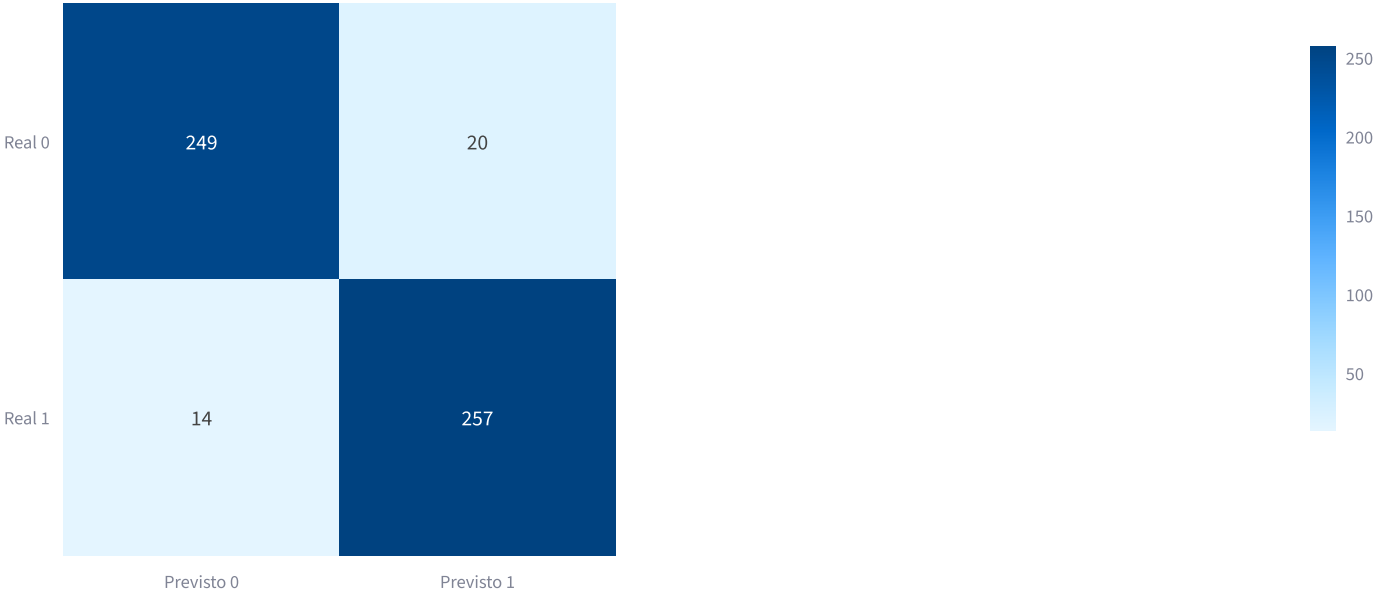
O maior número da tabela não escolhe o modelo sozinho. Também importam estabilidade, custo do erro, interpretação e manutenção.

Matriz de confusão — onde exatamente o modelo erra?

Threshold usado na matriz de confusão:

0.50

Matriz de confusão — threshold=0.50



VP

257

Positivo corretamente identificado.

FP

20

Modelo acusou positivo, mas era negativo.

FN

14

Modelo deixou escapar um positivo.

VN

249

Negativo corretamente identificado.

Accuracy, Precision, Recall e F1

Métrica	Pergunta simples	Quando observar
Accuracy	Qual percentual total foi acertado?	Base razoavelmente balanceada.
Precision	Quando digo positivo, quantas vezes estou certo?	Falso positivo é caro.
Recall	Dos positivos reais, quantos encontrei?	Falso negativo é caro.
Specificity	Dos negativos reais, quantos reconheci?	Preservar a classe negativa é importante.
F1	Como equilibrar Precision e Recall?	Queremos equilíbrio entre Precision e Recall.
Balanced Accuracy	Como dar peso semelhante às classes?	Classes desbalanceadas.

SEÇÃO 08

Threshold — a probabilidade não é a decisão

? Pergunta

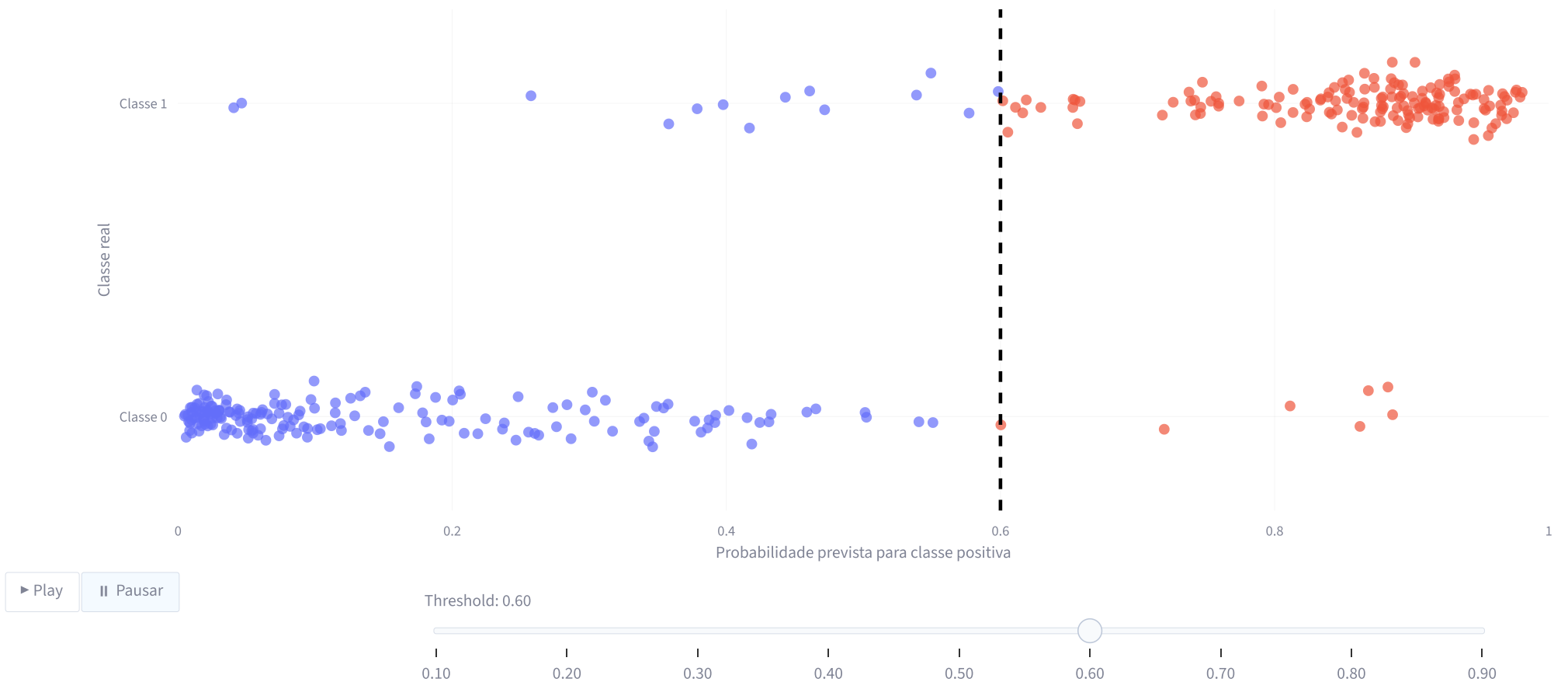
Por que 0.50 não precisa ser sempre o ponto de corte?

Muitos classificadores produzem uma probabilidade.

Um valor `0.73` só vira classe positiva porque escolhemos um threshold.

Mudar o threshold muda o equilíbrio entre **Precision** e **Recall**.

Threshold = 0.60 | Precision = 0.95 | Recall = 0.93



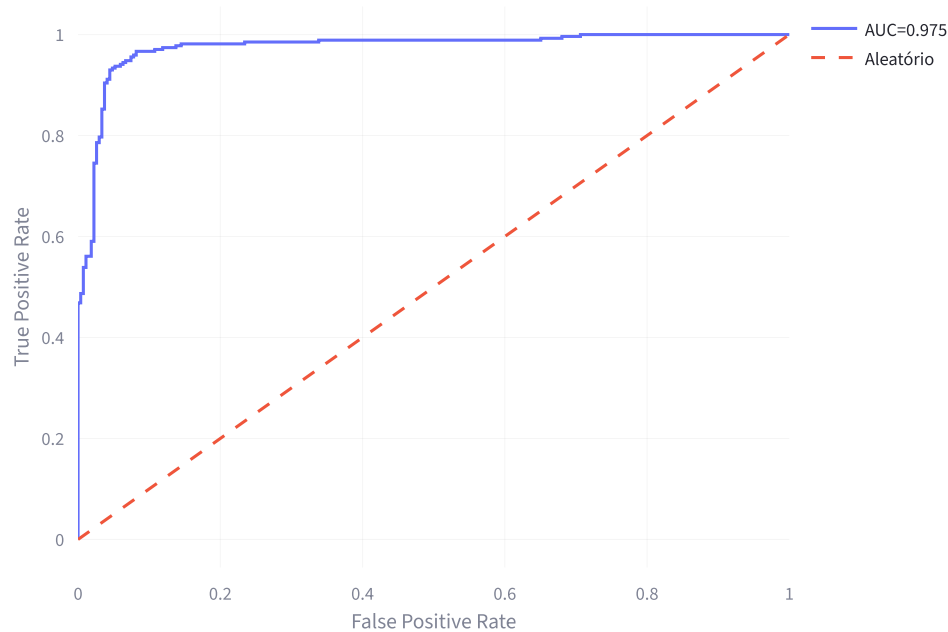
Na animação:

- azul = previsão negativa;
- vermelho = previsão positiva;
- linha pontilhada = threshold.

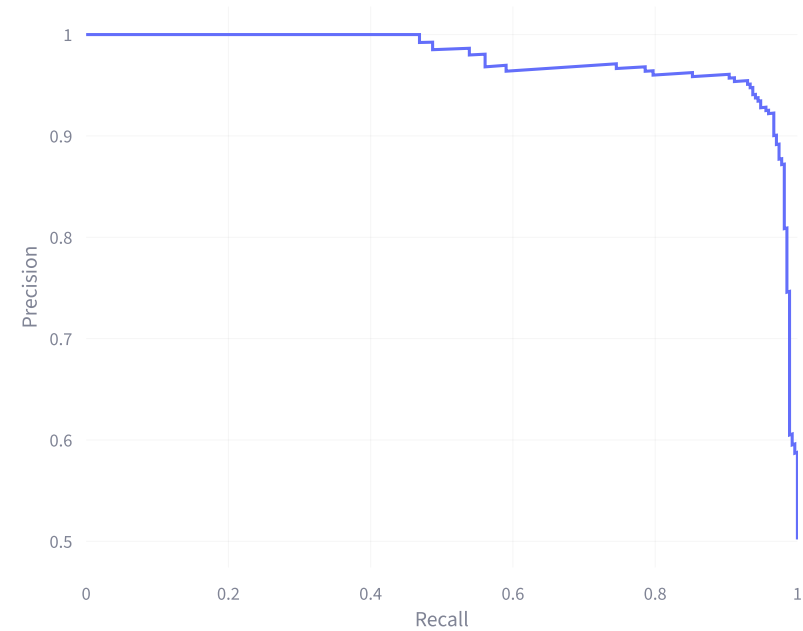
Conforme o threshold sobe, o modelo fica mais exigente para declarar positivo.

ROC Curve e Precision-Recall Curve

ROC Curve



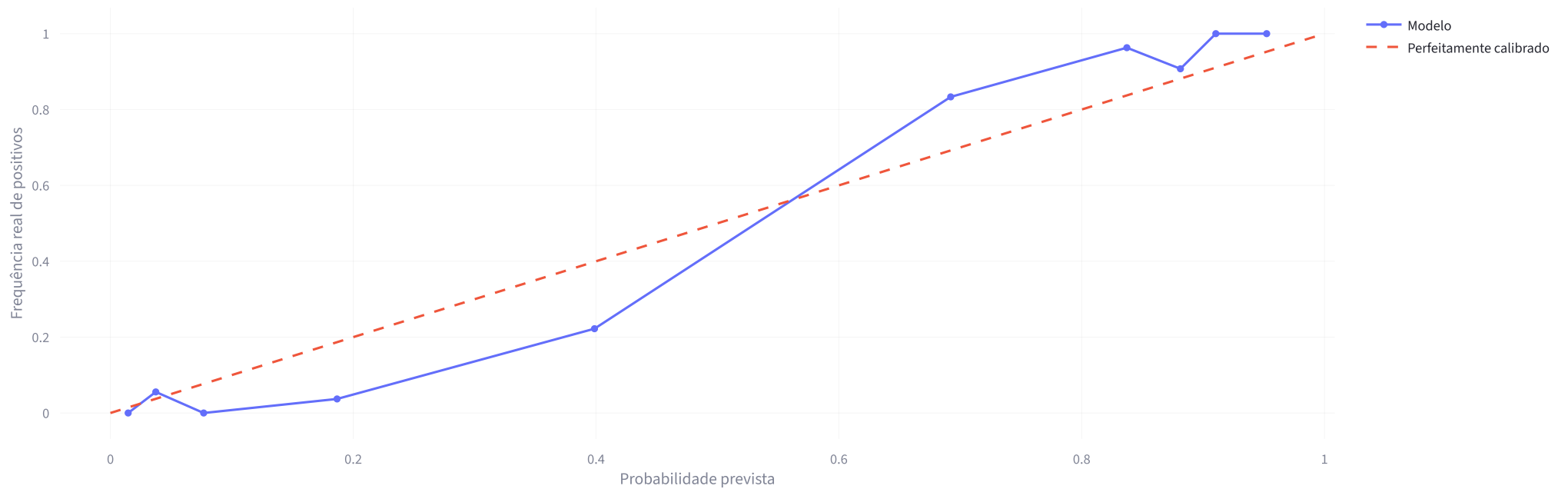
Precision-Recall Curve



Em classes raras, a **Precision-Recall Curve** é especialmente importante porque foca no comportamento da classe positiva.

Calibration — 80% deveria significar aproximadamente 80%

Calibration Curve



Calibration importa quando a própria probabilidade será usada para risco, priorização ou decisão.

SEÇÃO 11

O efeito do desbalanceamento

ACCURACY

0.939

Pode parecer excelente porque a classe 0 domina.

RECALL CLASSE 1

0.159

Quantos positivos reais foram encontrados.

F1 CLASSE 1

0.259

Equilibra Precision e Recall.

PR-AUC

0.502

Útil para avaliar a classe rara ao variar threshold.

Como ajudar a classe minoritária?

Class Weight

MUDAR O CUSTO DO ERRO

A base permanece igual, mas errar a classe rara passa a custar mais.

Oversampling

AUMENTAR A MINORIA

Repete observações minoritárias para equilibrar o treino.

Undersampling

REDUZIR A MAIORIA

Descarta parte da classe majoritária. Pode perder informação.

SMOTE

GERAR PONTOS SINTÉTICOS

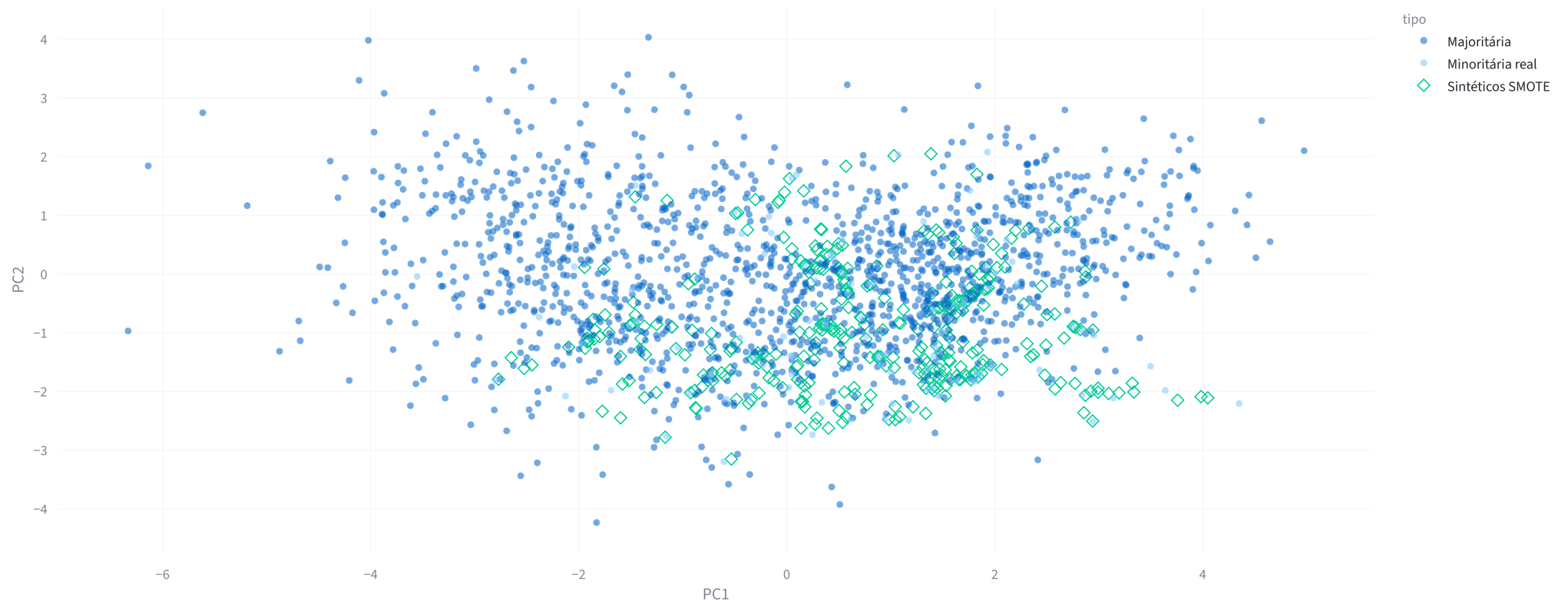
Cria observações entre exemplos minoritários próximos.

Estratégia	Accuracy	Balanced Accuracy	Precision	Recall	F1	PR-AUC
Sem tratamento	0.939	0.577	0.700	0.159	0.259	0.502
class_weight='balanced'	0.800	0.819	0.228	0.841	0.359	0.470
Random Oversampling	0.798	0.818	0.227	0.841	0.357	0.471
Random Undersampling	0.771	0.804	0.204	0.841	0.329	0.431
SMOTE simplificado	0.815	0.806	0.236	0.795	0.365	0.480

O SMOTE deste treinamento é uma implementação didática simplificada. Em produção, use implementações consolidadas como imbalanced-learn.

O que o SMOTE tenta fazer?

Base antes do SMOTE — projeção PCA



SEÇÃO 14

Parâmetros dos modelos que ajudam classes minoritárias

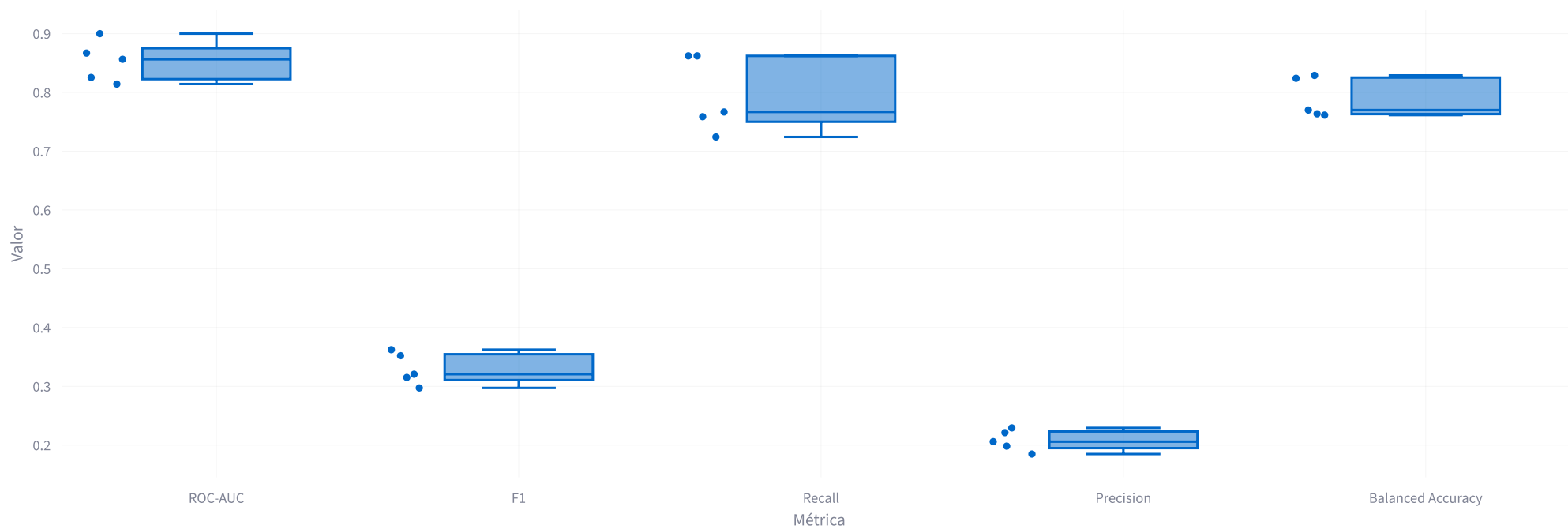
Modelo / família	Parâmetro comum	Ideia
LogisticRegression	class_weight	Aumentar o peso do erro da classe rara.
DecisionTree / RandomForest	class_weight	Aplicar pesos ao construir árvores.
SVM	class_weight	Penalizar classes de forma diferente.
XGBoost	scale_pos_weight / sample_weight	Dar mais peso à classe positiva.
LightGBM	class_weight / scale_pos_weight	Ajustar peso relativo das classes.
CatBoost	class_weights	Definir pesos por classe.

SEÇÃO 15

Um único split pode enganar

Fold	ROC-AUC	F1	Recall	Precision	Balanced Accuracy
1	0.900	0.362	0.862	0.229	0.829
2	0.814	0.297	0.759	0.185	0.761
3	0.825	0.321	0.724	0.206	0.764
4	0.867	0.352	0.862	0.221	0.824
5	0.856	0.315	0.767	0.198	0.770

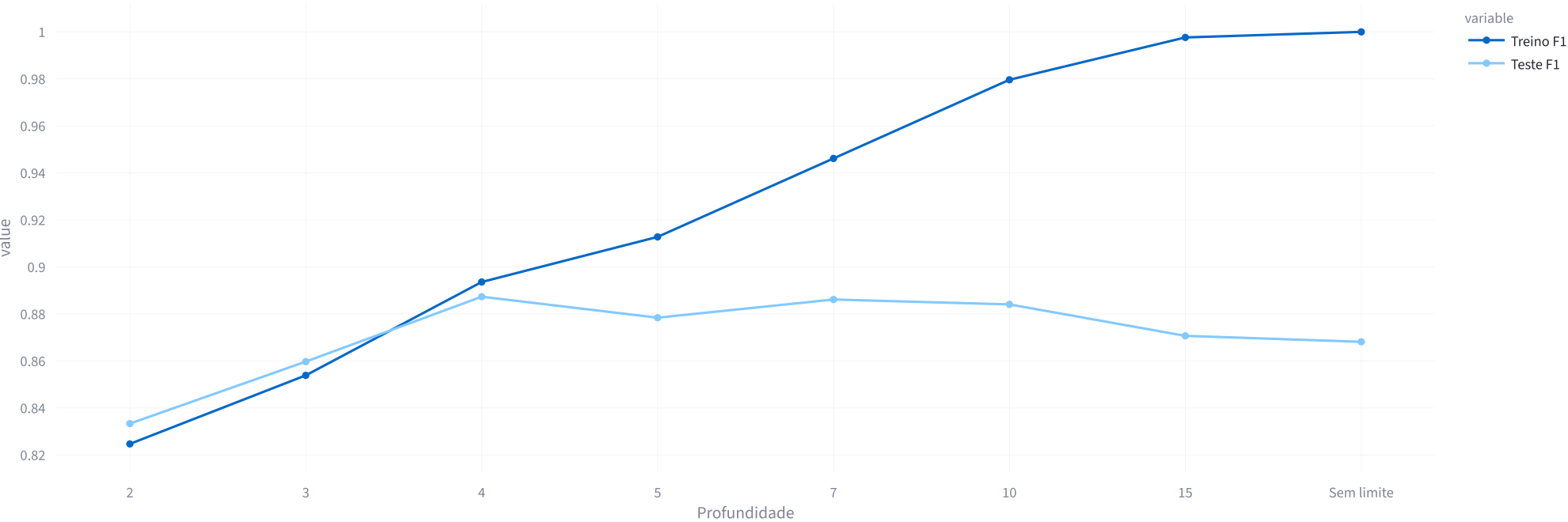
Variação das métricas entre folds



SEÇÃO 16

Overfitting — ótimo no treino, ruim fora dele

Complexidade da árvore × performance



SEÇÃO 17

Otimização de hiperparâmetros

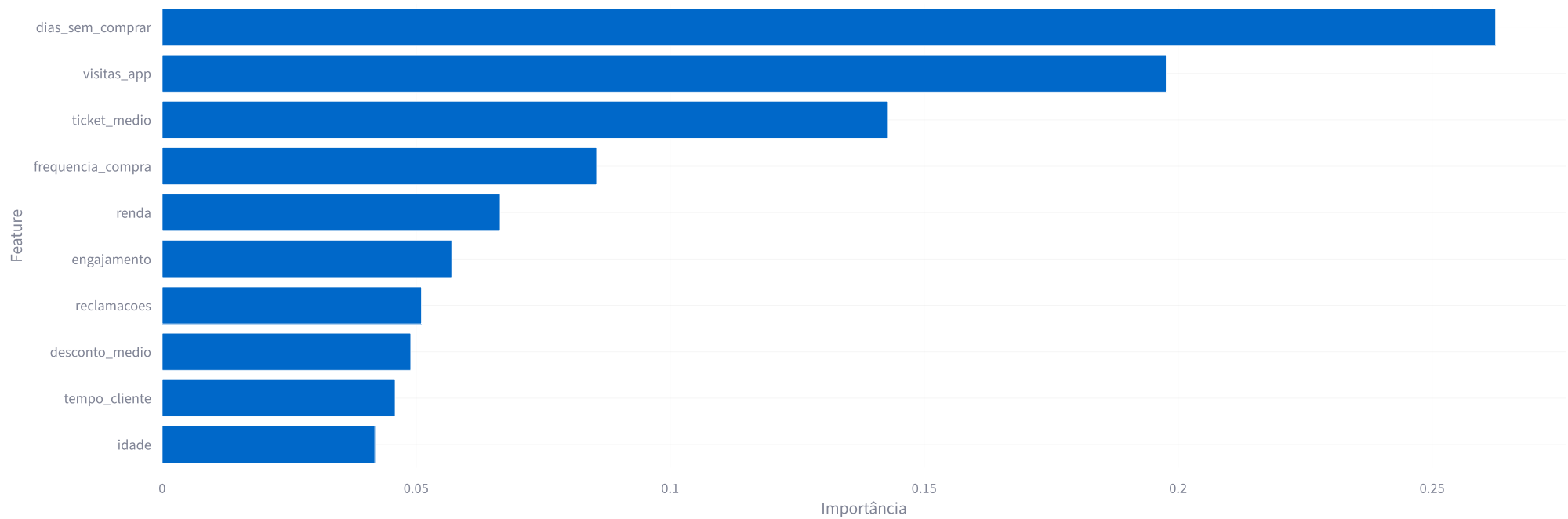
Método	Como funciona	Quando é útil
Grid Search	Testa todas as combinações definidas.	Espaço pequeno.
Random Search	Sorteia combinações no espaço de busca.	Espaços maiores.
Bayesian Optimization	Usa resultados anteriores para escolher os próximos testes.	Treinos caros.
Optuna	Framework de otimização com busca inteligente e pruning.	Projetos mais robustos.

rank_test_score	mean_test_score	std_test_score	param_n_estimators	param_max_depth	param_min_samples_split	param_min_samples_leaf	param_max_features
1	0.4857	0.0202	450	None	8	8	None
2	0.4689	0.0194	300	None	12	4	log2
3	0.462	0.0236	200	None	12	8	log2
4	0.4573	0.0323	300	8	12	2	None
5	0.4554	0.0491	450	None	8	2	None
6	0.4519	0.03	200	8	12	2	log2
7	0.4424	0.0387	300	6	12	2	None
8	0.4219	0.0361	120	8	2	2	log2
9	0.4195	0.0484	300	6	12	1	None
10	0.4177	0.0104	450	6	4	2	sqrt

Melhor F1 médio na busca: 0.486

Feature Importance — o que o modelo está utilizando?

Feature Importance nativa do Random Forest



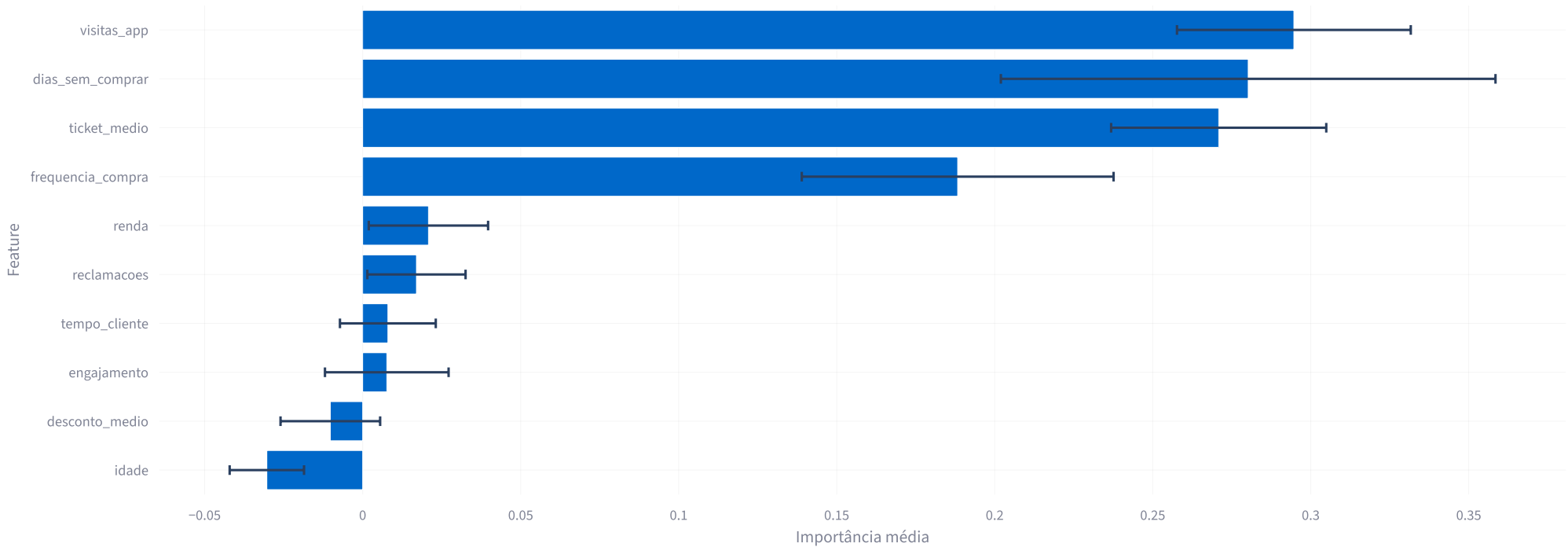
Feature importance ajuda a entender **quais variáveis o modelo mais utiliza**.

Mas atenção:

- importância não significa causalidade;
- duas features correlacionadas podem dividir importância;
- uma feature muito importante pode até revelar leakage.

Permutation Importance — o quanto a performance cai se bagunçarmos uma feature?

Permutation Importance no conjunto de teste



Procedimento:

1. medimos a performance normal;
2. embaralhamos uma feature;
3. medimos novamente;
4. se a performance cai muito, a feature era importante.

SEÇÃO 20

SHAP — explicando contribuições das features

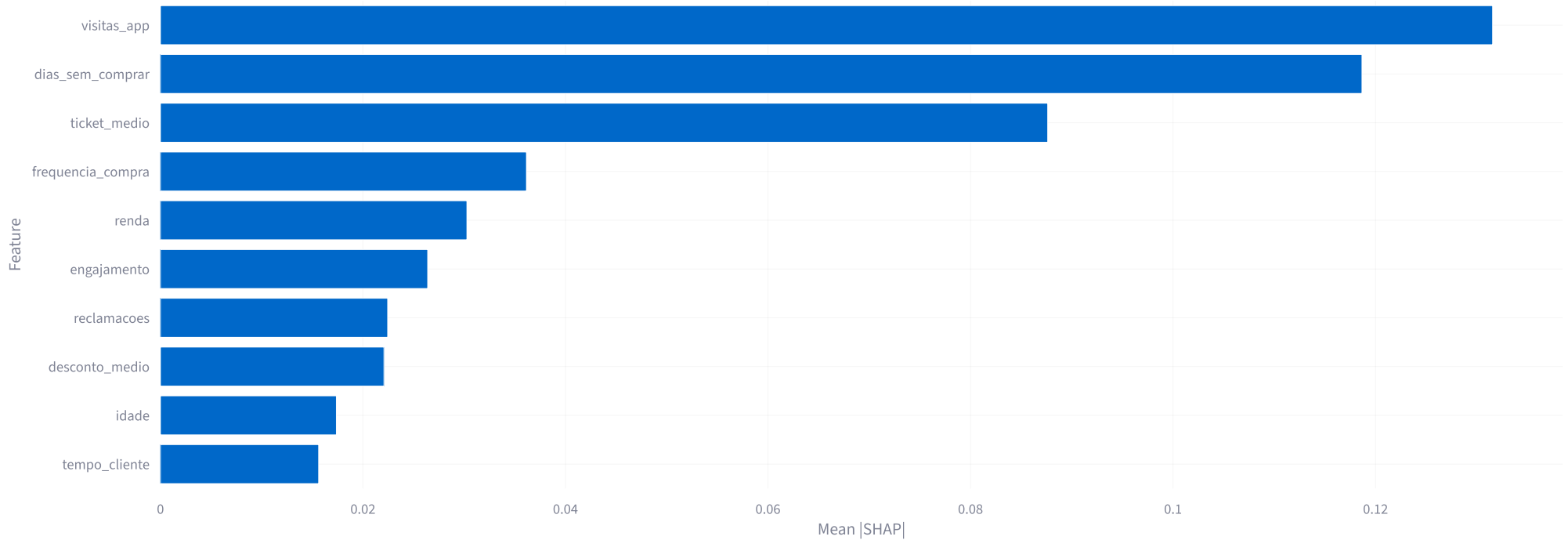
Uma forma simples de pensar em SHAP:

previsão = valor base + contribuições das features

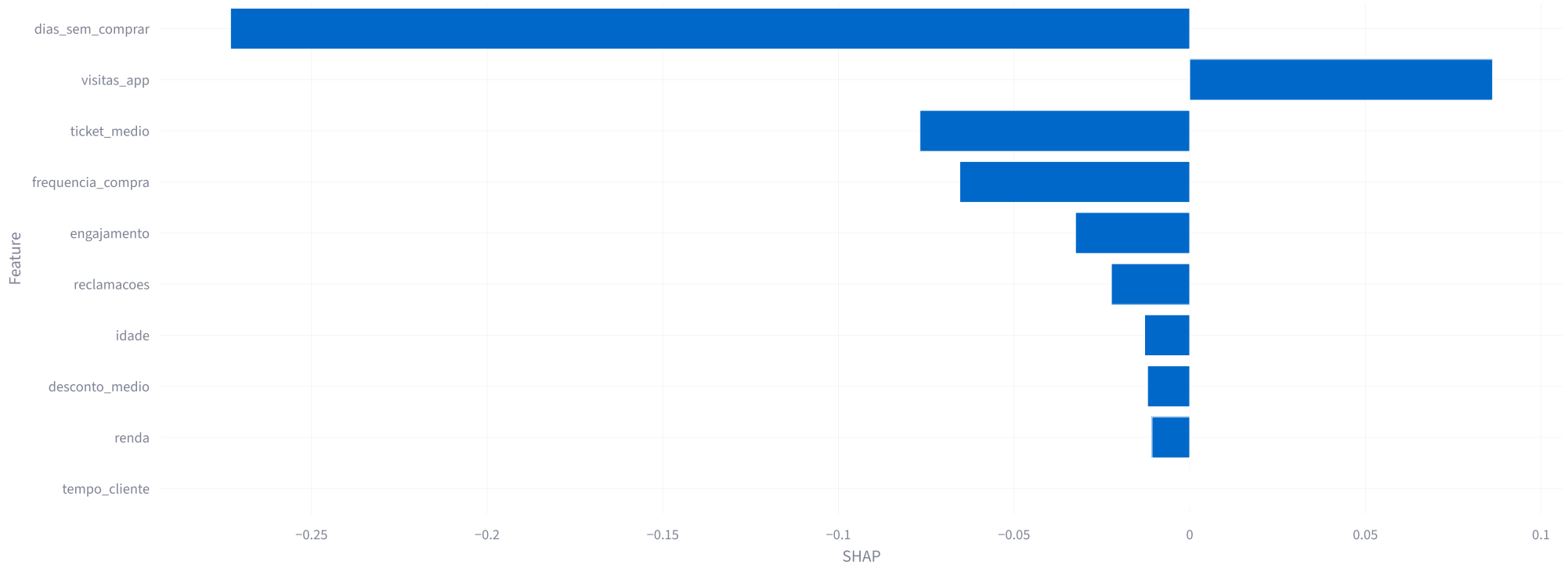
Isso permite duas leituras:

- **global:** quais features mais influenciam o modelo?
- **local:** por que esta observação recebeu esta previsão?

Importância global aproximada por SHAP



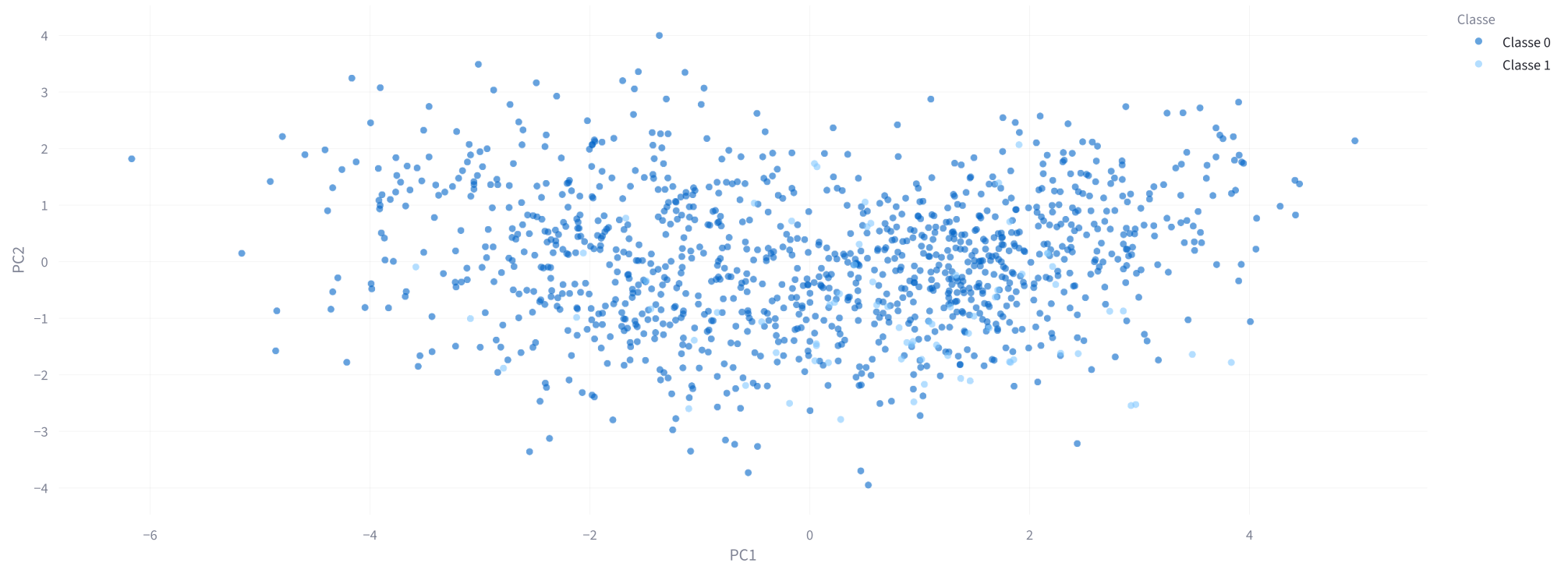
Explicação local — contribuição das features



SEÇÃO 21

Redução de dimensionalidade — PCA e visualização

PCA — projeção de 10 features em 2 dimensões



VARIÂNCIA PC1

39.5%

Variabilidade capturada pelo primeiro componente.

VARIÂNCIA PC2

15.5%

Variabilidade capturada pelo segundo componente.

TOTAL MOSTRADO

55.0%

Variância representada nessa projeção 2D.

PCA pode ajudar em visualização, compressão e redução de colinearidade.

Mas PCA não melhora um modelo automaticamente.

UMAP e t-SNE também são muito usados para visualizar estruturas em alta dimensão.

Antes de dizer que o modelo está pronto

Pergunta	Por que importa
O split representa o uso real?	Split errado produz avaliação irreal.
Existe Data Leakage?	Leakage infla artificialmente a performance.
A classe minoritária está sendo avaliada?	Accuracy pode esconder falhas graves.
A métrica corresponde ao custo do erro?	Nem todo erro tem o mesmo custo.
O modelo é estável entre folds?	Resultados instáveis são difíceis de confiar.
Treino e validação estão próximos?	Diferença grande pode indicar overfitting.
O threshold foi escolhido conscientemente?	0.50 é apenas um padrão.
As probabilidades precisam ser calibradas?	Probabilidade ruim prejudica decisões de risco.
A performance é consistente entre segmentos?	Média global pode esconder grupos problemáticos.
As features importantes fazem sentido?	Importância estranha pode indicar leakage ou viés.

Classificação multirrótulo

? Pergunta

E se uma observação puder pertencer a várias classes ao mesmo tempo?

Comentário	Atendimento	Preço	Produto	Entrega	Loja	Pagamento
Loja limpa, mas atendimento demorou.	1	0	0	0	1	0
Produto ótimo e preço bom.	0	1	1	0	0	0
Entrega atrasou e pagamento falhou.	0	0	0	1	0	1

Diferente do multiclasse, as labels não competem entre si.

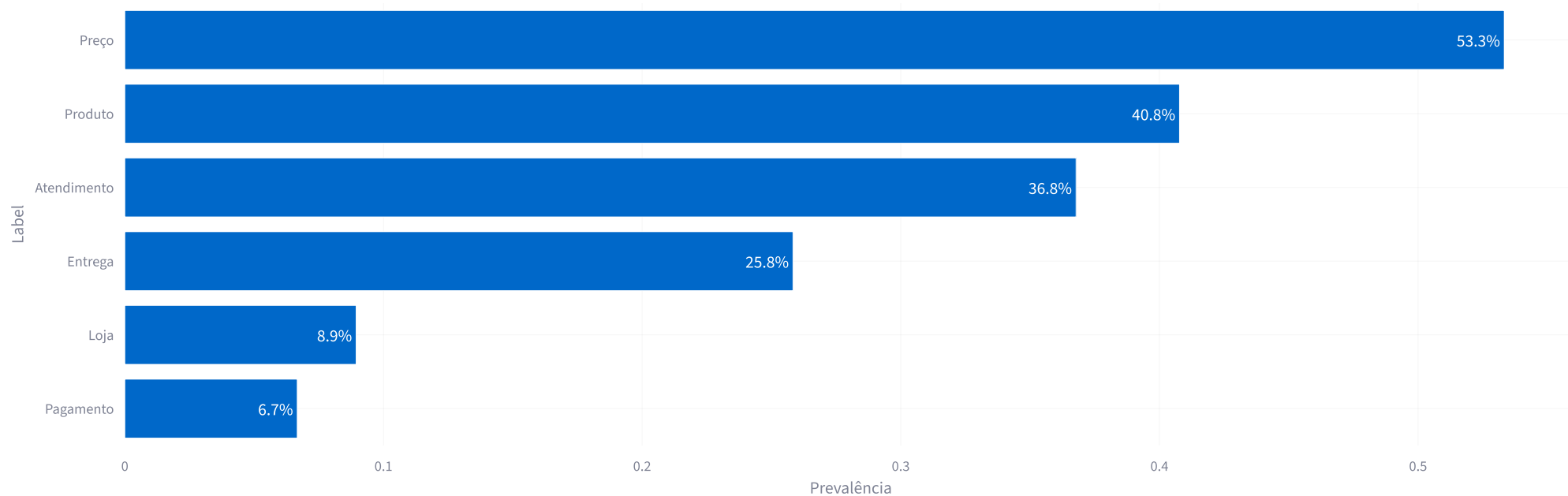
Um comentário pode ativar simultaneamente:

Atendimento = 1

Loja = 1

Desbalanceamento também existe por label

Prevalência das labels



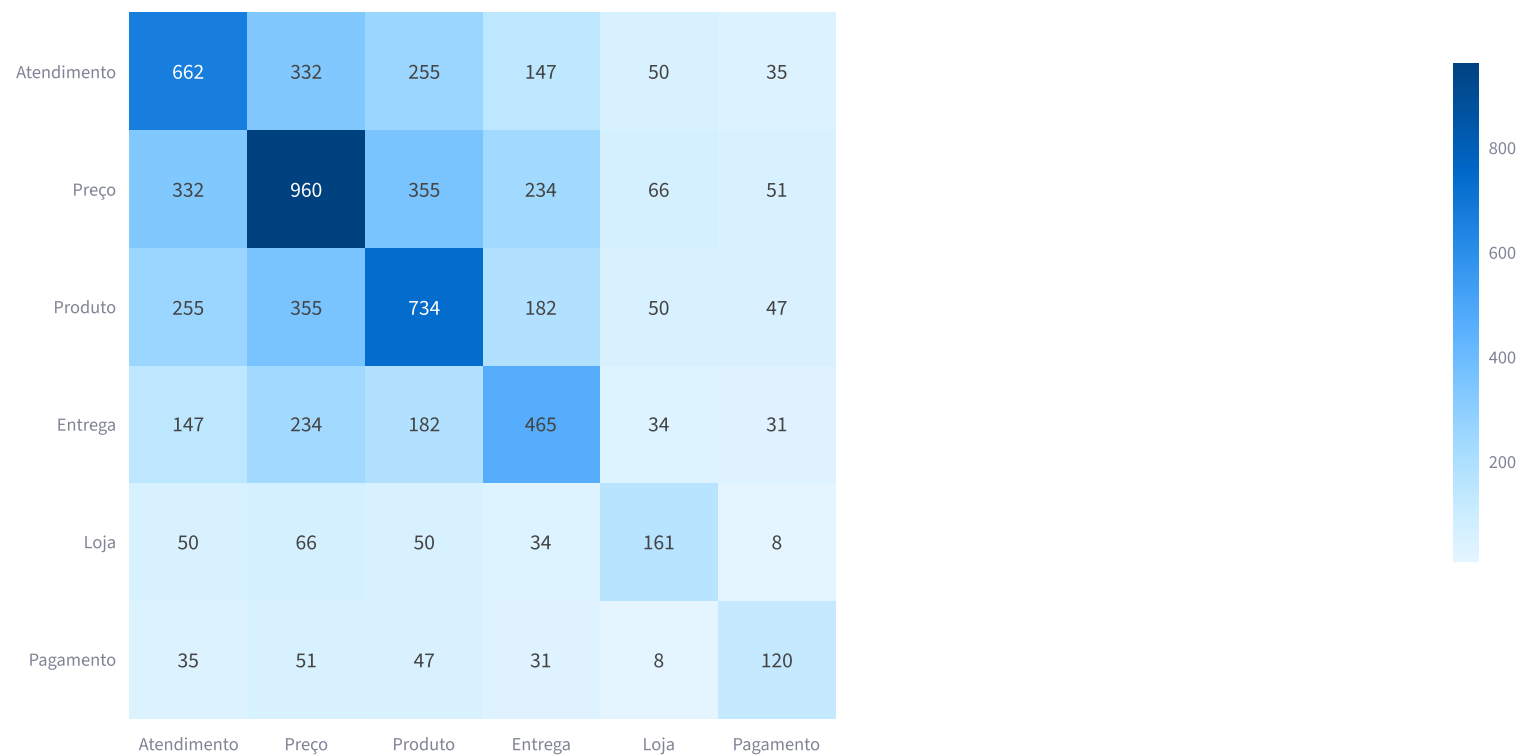
Em multilabel, algumas classes podem ser frequentes e outras raríssimas.

Por isso uma métrica agregada pode esconder problemas nas labels minoritárias.

SEÇÃO 25

Coocorrência — quais labels aparecem juntas?

Coocorrência entre labels



Se duas labels aparecem frequentemente juntas, pode existir informação útil na dependência entre elas.

Isso motiva técnicas como **Classifier Chains**.

One-vs-Rest

UM CLASSIFICADOR POR LABEL

Cada label é tratada como um problema binário independente. É um ótimo baseline.

Classifier Chain

LABELS AJUDAM A PREVER LABELS

As previsões anteriores entram como informação nas próximas labels.

SEÇÃO 27

Métricas para classificação multirrótulo

F1 MICRO

0.625

Labels frequentes pesam mais.

F1 MACRO

0.560

Cada label recebe o mesmo peso.

F1 SAMPLES

0.641

Avalia o conjunto de labels por observação.

HAMMING LOSS

0.255

Percentual de decisões de label incorretas.

Label	Prevalência	Precision	Recall	F1	AP
Atendimento	38.3%	0.794	0.802	0.798	0.894
Preço	51.3%	0.796	0.733	0.763	0.787
Produto	39.3%	0.691	0.759	0.724	0.708
Entrega	25.7%	0.427	0.698	0.530	0.459
Loja	9.3%	0.197	0.740	0.311	0.393
Pagamento	6.3%	0.140	0.706	0.234	0.242

Exact Match / Subset Accuracy: 0.252

Micro: favorece labels frequentes.

Macro: cada label vale o mesmo, por isso é importante para classes raras.

Samples: avalia por registro.

Exact Match: só acerta quando todas as labels da observação estão corretas.

SEÇÃO 28

Um threshold para cada label

Label	Threshold padrão	Threshold otimizado para F1
Atendimento	0.50	0.54
Preço	0.50	0.38
Produto	0.50	0.44
Entrega	0.50	0.52
Loja	0.50	0.80
Pagamento	0.50	0.62

Configuração	F1 micro	F1 macro	Hamming Loss
Threshold = 0.50	0.625	0.560	0.255
Threshold por label	0.702	0.607	0.186

Neste treinamento os thresholds foram otimizados no conjunto mostrado apenas para demonstração. Em projeto real, escolha thresholds na validação e avalie uma única vez no teste.

SEÇÃO 29

Comparando One-vs-Rest e Classifier Chain

Modelo	F1 micro	F1 macro	Exact Match	Hamming Loss
One-vs-Rest	0.625	0.560	0.252	0.255
Classifier Chain	0.616	0.550	0.244	0.260

 Ideia principal

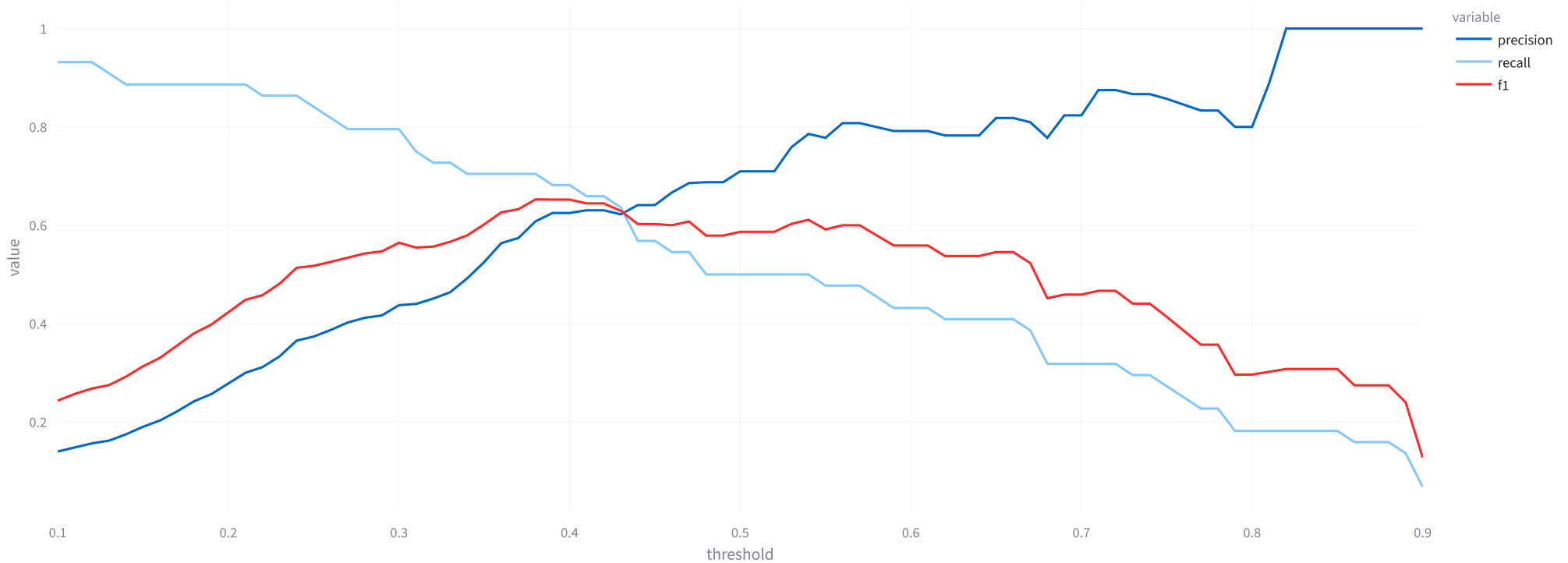
Classifier Chain não é automaticamente melhor. Dependências podem ajudar, mas também podem propagar erros.

SEÇÃO 30

Case final — Churn com classe minoritária

- 1. Entender a target
- 2. Separar treino / validação / teste
- 3. Criar baseline
- 4. Escolher métricas
- 5. Comparar modelos
- 6. Avaliar classe minoritária
- 7. Comparar class_weight / sampling
- 8. Ajustar threshold
- 9. Fazer cross-validation
- 10. Otimizar hiperparâmetros
- 11. Interpretar features
- 12. Avaliar estabilidade

Precision, Recall e F1 ao variar o threshold



THRESHOLD ESCOLHIDO

0.38

Aqui escolhido pelo maior F1.

PRECISION

0.608

Entre positivos previstos, quantos eram positivos reais.

RECALL

0.705

Entre positivos reais, quantos foram encontrados.

F1

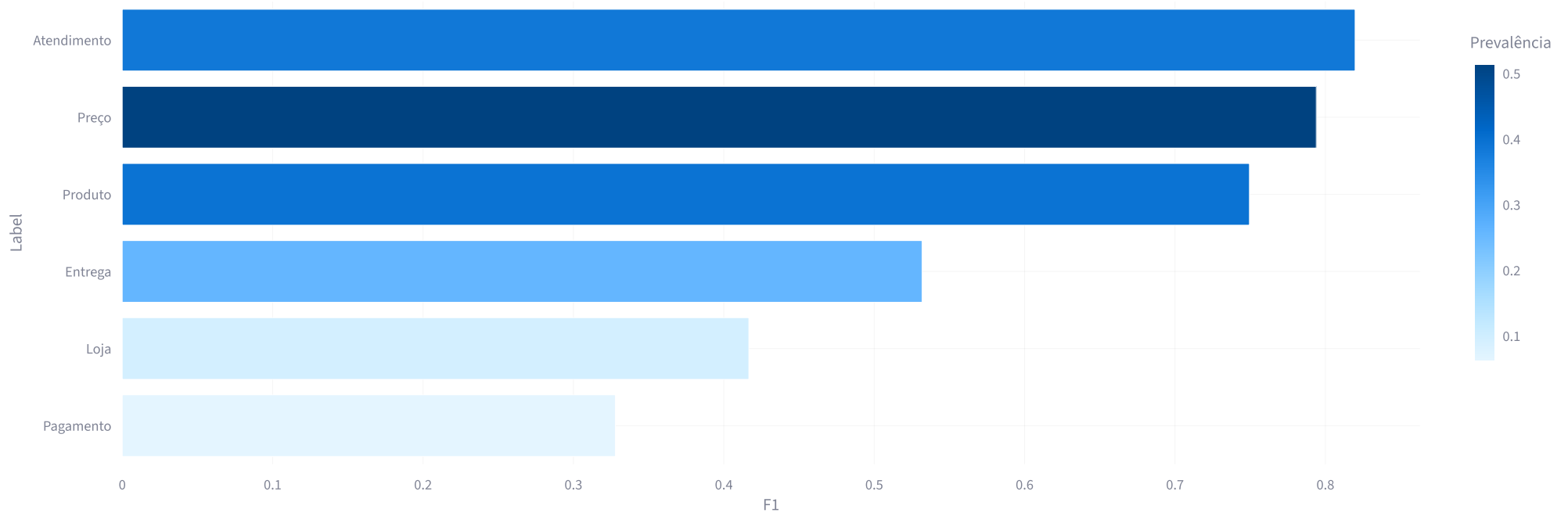
0.653

Equilíbrio entre Precision e Recall.

1. Definir a matriz multilabel
2. Avaliar prevalência por label
3. Avaliar coocorrência
4. Criar baseline One-vs-Rest
5. Avaliar F1 micro e macro
6. Avaliar métricas por label
7. Investigar classes raras
8. Comparar Classifier Chain
9. Ajustar threshold por label
10. Avaliar Exact Match / Hamming Loss
11. Interpretar erros por label

Label	Prevalência	Threshold	Precision	Recall	F1	AP
Atendimento	38.3%	0.54	0.838	0.802	0.820	0.894
Preço	51.3%	0.38	0.746	0.848	0.794	0.787
Produto	39.3%	0.44	0.668	0.854	0.749	0.708
Entrega	25.7%	0.52	0.444	0.662	0.532	0.459
Loja	9.3%	0.80	0.682	0.300	0.417	0.393
Pagamento	6.3%	0.62	0.227	0.588	0.328	0.242

F1 por label — observe especialmente as classes raras



💡 Ideia principal

Em multilabel, nunca encerre a análise olhando apenas para F1 micro. Sempre verifique performance por label e classes minoritárias.

SEÇÃO 32

O que levar deste treinamento?

1. Accuracy pode ser enganosa

Em bases desbalanceadas, um modelo pode ter accuracy alta e ignorar a classe rara.

2. Métrica depende do custo do erro

Precision, Recall, F1, ROC-AUC e PR-AUC respondem perguntas diferentes.

3. Threshold é uma decisão

0.50 é apenas um padrão.

4. Balanceamento é uma estratégia

Class weights, oversampling, undersampling, SMOTE e threshold precisam ser comparados em validação.

5. Tuning não corrige problema de dados

Hiperparâmetros não resolvem leakage, target ruim ou features inadequadas.

6. Interpretação faz parte da validação ⇔

Feature Importance, Permutation Importance e SHAP ajudam a entender o que o modelo aprendeu.

7. Multilabel exige análise por label

Micro F1 pode esconder labels raras ruins.

8. Um modelo só está pronto quando a avaliação representa a realidade

Performance, estabilidade, custo dos erros, calibration, segmentos e interpretação precisam fazer sentido juntos.

Fim do treinamento de Classificação.